CG3207 Assignment 3 — Study Guide from Zero

Everything you need for the Week 9 demo, explained from the very beginning

This guide assumes you know nothing. It starts with what a "bit" is and builds up, one idea at a time, until you can explain every part of our Assignment 3 design and answer the quiz questions about Booth's algorithm. Every chapter ends with a short recap and exercises (answers are at the end of each chapter). Read it in order: each chapter uses the ideas of the ones before.

Part	Chapters	What you'll be able to do afterwards
1 — Foundations	1 Binary numbers · 2 Negative numbers · 3 Digital hardware · 4 FPGAs and Verilog · 5 The processor	Read any number in binary or hex, understand registers, adders, clocks, state machines, read Verilog, and know how our CPU runs a program
2 — Multiply and divide	6 Multiplication · 7 Booth radix-2 · 8 Radix-4 and beyond · 9 Division	Multiply and divide in binary by hand exactly the way the hardware does, and explain why Booth works
3 — Our design	10 The MCycle unit · 11 Into the CPU · 12 The demo program · 13 Testing · 14 The visualisation · 15 Results and slides	Explain every file we changed, every number on the slides, and every test
4 — The demo	16 Demo script · 17 Question bank · 18 Practice problems · 19 Glossary · 20 Cheat sheet	Walk in and present with confidence

If you only have a few hours: Chapters 1–2 (numbers), 6–7 (multiplication and Booth), 10–11 (how it runs), 15–17 (results, demo, questions). Chapter 7 is the most important chapter in this guide.

PART 1 — FOUNDATIONS

Chapter 1 — Bits and binary numbers

1.1 What a bit is

Computers are built from switches that are either **off** or **on**. We write off as **0** and on as **1**. One such 0-or-1 value is a **bit** (binary digit). Everything a computer stores — numbers, letters, instructions — is a row of bits.

1.2 Counting in binary

In everyday **decimal** (base 10), each position is worth 10 times the one to its right: 1s, 10s, 100s, 1000s. The number 352 means 3×100 + 5×10 + 2×1.

Binary (base 2) works the same way, but each position is worth **2** times the one to its right: 1, 2, 4, 8, 16, 32, …, and each digit can only be 0 or 1.

Position (from the right)	7	6	5	4	3	2	1	0
Weight	128	64	32	16	8	4	2	1

So **1101** means 1×8 + 1×4 + 0×2 + 1×1 = **13**.

Counting up in 4 bits:

Decimal	Binary	Decimal	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

With **n bits** you can write 2^n different patterns. 4 bits: 16 patterns (0–15). 8 bits: 256. **32 bits** (the size of numbers in our CPU): 4,294,967,296.

Bit names: the rightmost bit is **bit 0**, the **least significant bit (LSB)**. The leftmost is the **most significant bit (MSB)**: bit 3 in a 4-bit number, bit 31 in a 32-bit number. We write a group of bits as, e.g., `Q[3:0]` = bits 3 down to 0 of Q, and `Q[0]` = just bit 0.

1.3 Converting decimal to binary

Repeatedly take away the biggest power of 2 that fits. Example, 22:

- 16 fits (22 − 16 = 6) → bit 4 = 1
- 8 doesn't fit → bit 3 = 0
- 4 fits (6 − 4 = 2) → bit 2 = 1
- 2 fits (2 − 2 = 0) → bit 1 = 1

- 1 doesn't fit → bit 0 = 0

22 = **10110**.

1.4 Hexadecimal: a short way to write binary

Long binary strings are hard to read, so we group them into **4 bits at a time** and write each group as one **hexadecimal** (base 16) digit. Hex needs 16 digit symbols, so after 9 it uses letters:

Binary	Hex	Decimal	Binary	Hex	Decimal
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	A	10
0011	3	3	1011	B	11
0100	4	4	1100	C	12
0101	5	5	1101	D	13
0110	6	6	1110	E	14
0111	7	7	1111	F	15

A 32-bit number is exactly 8 hex digits. Example: `1111 1111 1111 1111 1111 1111 1110 1110` = `F F F F F F E E` = **`FFFFFFEE`**. Hex numbers are often written with `0x` in front: `0xFFFFFFEE`. The board's 7-segment display shows 8 hex digits, i.e. one 32-bit number.

1.5 Adding in binary

Same as decimal addition, column by column from the right, but 1 + 1 = 10 in binary (that's 2): write 0, **carry 1**.

```
    1 1         (carries)
  0 1 1 0       (6)
+ 0 0 1 1       (3)
---------
  1 0 0 1       (9)
```

Rules: 0+0 = 0; 0+1 = 1; 1+1 = 0 carry 1; 1+1+1 (with a carry in) = 1 carry 1.

1.6 Overflow and "modulo"

If the answer needs more bits than you have, the extra carry **falls off the end** and is lost. In 4 bits: 1111 (15) + 0001 (1) = 1 0000 → keep only 0000. The hardware is really computing "the answer modulo 2^n" (the remainder after dividing by 2^n). This sounds like a problem, but it's exactly what makes negative numbers work (Chapter 2).

1.7 Shifting

Shifting left by 1 moves every bit one place left and puts a 0 on the right: 0011 (3) → 0110 (6). That **multiplies by 2**. Shifting left by k multiplies by 2^k.

Shifting right by 1 moves every bit one place right: 0110 (6) → 0011 (3). That **divides by 2** (dropping any remainder). The bit that falls off the right is lost (unless the hardware saves it — Booth does).

Shifting is almost free in hardware: it's just connecting wire *i* to position *i+1*. Remember this: "multiplying by 2 is just a shift" is the reason radix-4 Booth is cheap.

Recap: binary = base 2, positions worth 1, 2, 4, 8...; hex = 4 bits per digit; adding carries like decimal; extra carries fall off (modulo 2^n); shift left = ×2, shift right = ÷2.

Exercises 1:

1. What is 1011 in decimal? 2. Write 19 in binary. 3. What is `0x2A` in decimal? 4. Add 0101 + 0111 in 4 bits. What is lost? 5. What is 0101 shifted left by 2?

Answers 1: 1. 8+2+1 = 11. 2. 16+2+1 → 10011. 3. 2×16 + 10 = 42. 4. 1100 (12); nothing lost (12 fits). 5. 10100 = 20 (= 5 × 4).

Chapter 2 — Negative numbers: two's complement

2.1 The problem

Binary as above only gives 0 and positive numbers ("**unsigned**"). We need negatives. The method every modern computer uses is **two's complement**.

2.2 The rule: the top bit is negative

In an n-bit two's complement number, the **top bit (MSB) has a negative weight**, -2^{n-1}. All other bits keep their normal positive weights.

For 4 bits the weights are **−8, 4, 2, 1**:

Bits	Calculation	Value
0000	0	0
0101	4 + 1	**5**
0111	4 + 2 + 1	**7** (largest)
1000	−8	**−8** (most negative)
1101	−8 + 4 + 1	**−3**
1110	−8 + 4 + 2	**−2**
1111	−8 + 4 + 2 + 1	**−1**

Key facts:

- If the top bit is 1, the number is negative. That's why the top bit is called the **sign bit**.
- **4-bit range: −8 to +7. 32-bit range: −2,147,483,648 (-2^{31}) to +2,147,483,647 ($2^{31} - 1$).**
- **There is one more negative number than positive.** −8 exists in 4 bits but +8 does not. **Remember this**: it's why our Booth registers need an extra bit.
- **All ones is −1**: 1111 in 4 bits, `FFFFFFFF` in 32 bits.

2.3 Why it's designed this way: addition just works

With two's complement, the same adder circuit adds positive and negative numbers correctly, as long as you ignore the carry that falls off. Example, 4 bits, 5 + (−3):

```
  0101   (5)
+ 1101   (-3)
------
1 0010   → drop the carry → 0010 = 2  ✓
```

So the hardware doesn't need separate circuits for negative numbers.

2.4 Negating a number: invert and add 1

To find −x: **flip every bit, then add 1.**

- −3: 3 = 0011 → flip → 1100 → +1 → **1101** ✓ (matches the table).
- −(−3): 1101 → flip → 0010 → +1 → 0011 = 3 ✓.
- −(−8): 1000 → flip → 0111 → +1 → 1000 = −8 again! **The most negative number cannot be negated in the same number of bits** (+8 doesn't exist in 4 bits). With 5 bits it works: −8 = 11000 → flip → 00111 → +1 → 01000 = +8 ✓.

Subtraction is addition of the negation: A − B = A + (flipped B) + 1. That's how hardware subtracts: invert B's bits and feed a 1 into the adder's "carry in". **One adder can both add and subtract.** (Booth's A − M uses exactly this.)

2.5 Reading a negative hex number

Negate it to find its size. `FFFFFFEE`: flip → `00000011`, +1 → `00000012` = 18, so `FFFFFFEE` = **−18**. Shortcut for small negatives: count down from `FFFFFFFF` (−1): `FFFFFFFE` = −2, `FFFFFFFD` = −3, … `FFFFFFEE` is 17 below `FFFFFFFF`, so −1 − 17 = −18.

2.6 Same bits, two meanings: signed vs unsigned

The bits `FFFFFFFF` are **−1** if read as signed, and **4,294,967,295** if read as unsigned. The bits don't say which — **the instruction decides**. That's why RISC-V has both `div` (signed) and `divu` (unsigned): on −6 ÷ 3, `div` gives −2 (`FFFFFFFE`), while `divu` reads −6 = `FFFFFFFA` as 4,294,967,290 and gives 1,431,655,763 (`55555553`).

2.7 Sign extension: making a number wider

To widen a **signed** number, **copy its sign bit** into all the new positions. 4-bit −3 = 1101 → 8-bit 1111 1101 (still −3: −128+64+32+16+8+4+1 = −3). Positive 0101 → 0000 0101.

To widen an **unsigned** number, fill with **0s** ("zero extension").

2.8 Arithmetic shift right

Shifting a **signed** number right must keep its sign. An **arithmetic shift right** (written `>>>`) copies the sign bit into the new top position:

- 1100 (−4) `>>>` 1 = **1110** (−2) ✓ — divided by 2.
- A plain ("logical") shift would give 0110 (+6) — wrong for signed numbers.

Booth's algorithm shifts its partial result this way.

Recap: top bit = -2^{n-1}; all ones = −1; one more negative than positive; negate = flip + 1; subtract = add the flip + 1; sign-extend by copying the top bit; arithmetic shift right keeps the sign.

Exercises 2:

1. What is 1010 as a signed 4-bit number? 2. Write −5 in 4 bits. 3. What is `FFFFFFF1`? 4. Sign-extend 1011 to 8 bits. 5. 1010 `>>>` 1 = ? 6. Can −8 be negated in 4 bits?

Answers 2: 1. −8 + 2 = −6. 2. 5 = 0101 → 1010 → 1011. 3. flip → 0E, +1 → 0F = 15 → −15. 4. 1111 1011. 5. 1101 (−6 → −3). 6. No: +8 doesn't fit; you need 5 bits.

Chapter 3 — Digital hardware: the building blocks

3.1 Logic gates

A **gate** is a tiny circuit computing a simple function of bits:

Gate	Output is 1 when…	Verilog
NOT	the input is 0	`~a`
AND	both inputs are 1	`a & b`
OR	either input is 1	`a | b`
XOR	the inputs differ	`a ^ b`

XOR as a controllable inverter: `a ^ 0 = a`, `a ^ 1 = ~a`. So XOR-ing every bit of M with a "negate" signal gives either M or its flipped version. Radix-4 Booth uses this.

3.2 The multiplexer (mux): choosing between inputs

A **mux** has several data inputs and a **select** input; it passes one input through, chosen by select. A 2-to-1 mux: `out = sel ? b : a` ("if sel is 1, b, else a"). Our CPU uses muxes everywhere to choose where values come from (e.g. "write the ALU's result or MCycle's result?").

3.3 The adder

An **adder** adds two n-bit numbers. The simplest kind (**ripple-carry**) chains n one-bit adders: each column's carry goes into the next column. The carry may have to travel through every column, so **a wider adder is slower**: a 64-bit adder takes longer to produce its answer than a 33-bit one. With a "carry in" of 1 and B inverted, the same adder subtracts (Chapter 2.4).

3.4 Combinational logic

Gates, muxes and adders are **combinational**: their output depends only on their current inputs, and changes (after a short delay) whenever the inputs change. They have no memory.

3.5 Registers and the clock

To **remember** values, hardware uses **flip-flops**. A flip-flop stores one bit. A **register** is a group of flip-flops storing a multi-bit value (e.g. a 32-bit register = 32 flip-flops).

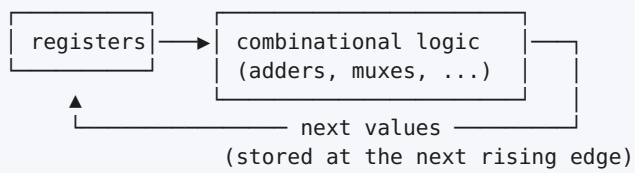
A register only changes at one instant: the **rising edge of the clock**. The **clock** is a signal that goes 0, 1, 0, 1, … steadily. At each rising edge (0→1), every register copies its input; between edges, registers hold their values no matter what their inputs do.

```
clock:   _|‾|_|‾|_|‾|_|‾|_|‾|_
              ↑   ↑   ↑      ← rising edges: registers update here
```

One period of the clock is one **clock cycle**.

3.6 How a synchronous circuit works

Almost all digital designs follow one pattern:



1. Registers hold the current values ("state").
2. Combinational logic computes the **next** values from them.
3. At the rising edge, the registers store those next values.
4. Repeat every cycle.

The clock period must be long enough for the slowest path through the combinational logic to finish. That slowest path is the **critical path**. If you put two adders one after the other, the critical path roughly doubles, and the clock must slow down. (This matters for the "how to get fewer cycles" question in Chapter 8.)

3.7 State machines (FSMs)

A **finite state machine** is a circuit that is always in one of a few named **states** and moves between them at clock edges depending on its inputs. MCycle has two states, **IDLE** and **COMPUTING**.

- A **Moore** machine's outputs depend only on its state.
- A **Mealy** machine's outputs also depend on its current inputs, so an output can change *immediately* when an input changes, without waiting for a clock edge. MCycle's **Busy** output is Mealy: it goes high in the same instant **Start** goes high. (Why that matters: Chapter 11.)

Recap: gates and muxes and adders = combinational (no memory, delay); registers = memory, updated only at rising clock edges; each cycle: logic computes next values, the edge stores them; the slowest path sets the clock period; an FSM moves between states each cycle.

Exercises 3:

1. What does `sel ? b : a` output when sel = 0? 2. What is `1010 ^ 1111`? 3. A register's input changes in the middle of a cycle. When does the register's value change? 4. Why is a 64-bit adder slower than a 33-bit one? 5. What's the difference between a Mealy and a Moore output?

Answers 3: 1. a. 2. 0101 (XOR with all ones flips). 3. At the next rising clock edge. 4. The carry may ripple through more columns. 5. A Mealy output can change as soon as an input changes; a Moore output only changes with the state (at clock edges).

Chapter 4 — FPGAs and Verilog

4.1 The FPGA

An **FPGA** (Field-Programmable Gate Array) is a chip full of configurable logic that you can "wire up" to become any digital circuit. Ours is a Xilinx Artix-7 (XC7A100T) on the **Nexys 4 DDR** board, which also has 16 switches, 5 push buttons, 16 LEDs and an 8-digit 7-segment display.

Inside, the FPGA is made of:

- **LUTs** (look-up tables): tiny memories that can implement any logic function of up to 6 inputs. All combinational logic becomes LUTs.
- **Flip-flops (FFs)**: the registers.
- **Carry chains** (CARRY4): fast dedicated wiring for adders.
- **DSP blocks**: hardware multipliers. The `*` operator in Verilog would use these — **forbidden** in this assignment, because the point is to build the multiplier ourselves.

When the slides say "650 LUTs, 365 flip-flops", that's how much of the chip our MCycle uses.

4.2 Verilog: describing hardware with text

Verilog is a hardware description language. It looks like a programming language, but it **describes a circuit**, not a sequence of steps. The pieces you need:

```
module MCycle #(parameter width = 4) (   // a module = a circuit block; parameters are settings
    input  CLK,                          // inputs and outputs = the block's pins
    input  [1:0] MCycleOp,               // [1:0] = a 2-bit bus, bits 1 down to 0
    output reg Busy
);
```

- `wire` = a connection; `reg` = something assigned inside an `always` block (it may or may not become a real register).
- `assign x = a & b;` = continuous combinational logic.
- `always @(*) begin … end` = a block of **combinational** logic. It's written like a little program with `if` / `case`, but it describes logic that is always active. It uses **blocking assignments** `=`, which take effect immediately, so later lines see the new value.
- `always @(posedge CLK) begin … end` = **registers**: everything assigned here is stored at the rising clock edge. It uses **non-blocking assignments** `<=`: all of them happen together at the edge.
- `{a, b}` = **concatenation**: glue bit-vectors side by side. `{2'b10, 3'b011}` = 10011.
- `{4{x}}` = **replication**: x repeated 4 times. `{4{Operand1[3]}}` = the sign bit copied 4 times (sign extension).
- `x[7:4]` = bits 7 to 4 of x (a "slice").
- `4'b1010` = a 4-bit binary constant; `32'h0040_0000` = a 32-bit hex constant.
- `case (sel) … endcase` = choose one of several actions by value — becomes a mux.
- `for` **loops** in synthesisable Verilog don't loop in time; they **copy the hardware** that many times.

4.3 Latches: the classic mistake

In an `always @(*)` block, if some signal isn't assigned on every possible path (e.g. an `if` with no `else`), the hardware has to "remember" its old value — creating a **latch**, an accidental memory that isn't controlled by the clock. Latches cause timing problems. **Fix:** give every signal a default value at the top of the block. MCycle does this (Chapter 10); the slides report **0 latches**.

4.4 From Verilog to the board: the Vivado flow

Step	What happens	Output
Behavioural simulation	Run the Verilog on the computer with a testbench; look at results/waveforms	Pass/fail, waveforms
Synthesis	Vivado turns Verilog into a netlist of LUTs and flip-flops	Utilisation report (LUTs, FFs), warnings
Post-synthesis simulation	Simulate that netlist to check it still behaves like the Verilog	Pass/fail
Implementation	Place the LUTs/FFs on the chip and route wires between them	Timing report
Generate bitstream	Create the configuration file	`.bit` file
Program device	Load the `.bit` into the FPGA	The board runs your design

A **testbench** is extra Verilog (not part of the hardware) that feeds inputs to your design and checks outputs. A **self-checking** testbench computes the expected answers itself and prints PASS/FAIL.

Timing terms (slide 5):

- **Setup slack**: how much earlier than needed the slowest signal arrives before the clock edge. Positive = the design meets the clock speed.
- **Hold slack**: whether signals stay stable long enough after the edge. After synthesis it's only an estimate (no real wires yet); it's fixed during implementation. Negative post-synthesis hold slack is normal.

Recap: FPGA = LUTs + flip-flops + carry chains + DSPs; Verilog describes circuits; `always @(*)` + `=` → combinational, `always @(posedge CLK)` + `<=` → registers; defaults prevent latches; simulate → synthesise → implement → bitstream.

Exercises 4:

1. What does `{2{2'b10}}` give? 2. Which kind of `always` block creates registers? 3. Why give every signal a default in `always @(*)`? 4. Why is `*` forbidden in this assignment? 5. What's the difference between behavioural and post-synthesis simulation?

Answers 4: 1. 1010. 2. `always @(posedge CLK)`. 3. To avoid creating latches. 4. It would use DSP multipliers instead of a multiplier we design. 5. Behavioural simulates the Verilog source; post-synthesis simulates the LUT/FF netlist synthesis produced.

Chapter 5 — The processor (what Assignments 2 and 3 built)

5.1 What a processor does

A **processor (CPU)** runs a **program**: a list of **instructions** stored in memory, each a 32-bit number. It repeats forever:

1. **Fetch** the instruction at the address in the **program counter (PC)**.
2. **Decode** it: work out what it asks for.
3. **Execute** it: read registers, compute, maybe access memory, write the result.
4. Move the PC to the next instruction (PC + 4, since each instruction is 4 bytes), or jump.

5.2 RISC-V

Our CPU implements **RISC-V** (RV32I), a real instruction set. It has **32 registers**, x0–x31, each 32 bits (x0 is always 0). They also have nicknames used in assembly: `a0`, `t0`, `s0`, etc. Examples:

- `add t0, a0, a1` → t0 = a0 + a1
- `lw t0, 8(s0)` → load a word from memory address s0 + 8 into t0
- `beq a0, a1, label` → jump to label if a0 = a1

Assignment 3 adds the "M" extension: `mul`, `mulh`, `mulhu`, `div`, `divu`, `rem`, `remu` (Chapter 11).

5.3 How an instruction is encoded

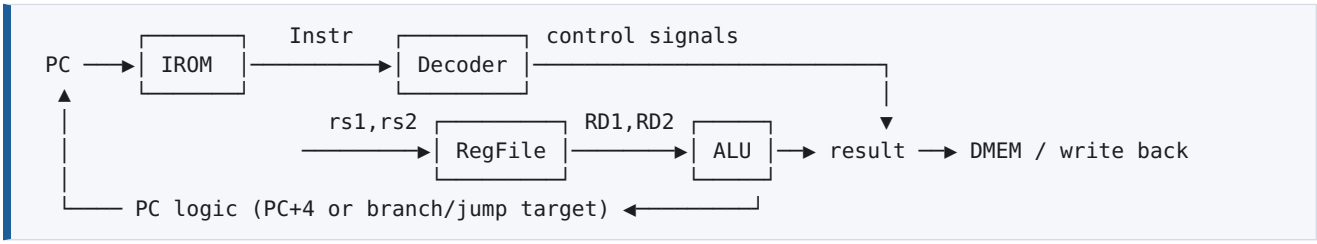
Each instruction is 32 bits, split into **fields**. For register-to-register ("R-type") instructions like `add` and `mul`:

Bits	31–25	24–20	19–15	14–12	11–7	6–0
Field	funct7	rs2	rs1	funct3	rd	opcode
Meaning	operation detail	2nd source reg	1st source reg	operation	destination reg	instruction type

`add`, `sub` and all the M instructions share the opcode **0110011**. They're told apart by funct7 and funct3: `add` has funct7 = 0000000, `sub` 0100000, and **all M instructions 0000001**.

5.4 Our single-cycle CPU (Assignment 2)

"Single-cycle" means **every instruction completes in one clock cycle**. Its blocks (each a Verilog module):



- **ProgramCounter**: the PC register.
- **IROM**: instruction memory (holds the program).
- **Decoder**: looks at opcode/funct3/funct7 and produces **control signals** (which ALU operation, write a register or not, read memory or not…).
- **RegFile**: the 32 registers; reads rs1 and rs2 (outputs RD1, RD2), writes rd.
- **ALU** (Arithmetic Logic Unit): add, subtract, and, or, shifts, compares.
- **DMEM**: data memory.
- **Wrapper**: connects the CPU to the memories and the board's peripherals.

The clock period must cover the slowest instruction — a load (`lw`): fetch → decode → read registers → ALU computes the address → read data memory → write back, all in one cycle.

5.5 Talking to the board: memory-mapped I/O

The switches, LEDs and display appear to the program as special memory addresses (base `0xFFFF0000`). Reading address `0xFFFF0064` reads the 16 switches; writing `0xFFFF0060` sets the LEDs; writing `0xFFFF0080` sets the 7-segment display; reading `0xFFFF00A0` reads a **cycle counter** that increases by 1 every clock cycle (our demo program uses it to measure how long `mul` takes).

The board's CPU clock is the 100 MHz board clock divided by 2^5 = **3.125 MHz**.

Recap: the CPU fetches, decodes, executes one instruction per cycle; instructions are 32-bit with opcode/funct3/funct7/rs1/rs2/rd fields; M instructions share add's opcode with funct7 = 0000001; the Decoder makes control signals; I/O is memory-mapped.

Exercises 5:

1. What does the PC hold? 2. Which field tells `mul` apart from `add`? 3. In a single-cycle CPU, what sets the clock period? 4. How does the program read the switches?

Answers 5: 1. The address of the current instruction. 2. funct7 (0000001 for M instructions). 3. The slowest instruction's path (a load). 4. By loading (lw) from the memory-mapped address 0xFFFF0064.

PART 2 — MULTIPLICATION AND DIVISION

Chapter 6 — How hardware multiplies

6.1 Long multiplication in decimal

To work out 123 × 45 by hand you multiply 123 by each digit of 45, shift each result left by its position, and add:

```
  123
x  45
-----
  615    (123 x 5)
 492     (123 x 4, shifted one place)
-----
 5535
```

6.2 Long multiplication in binary is easier

In binary each digit of the multiplier is 0 or 1, so each row is either **the multiplicand** or **nothing**. Multiplication becomes **"add shifted copies of the multiplicand wherever the multiplier has a 1"**.

Example, 13 × 6 (4-bit: 1101 × 0110):

```
      1101      multiplicand (13)
    x 0110      multiplier (6)
    ------
      0000      bit 0 = 0 → nothing
     1101       bit 1 = 1 → 13 shifted left 1
    1101        bit 2 = 1 → 13 shifted left 2
   0000         bit 3 = 0 → nothing
   --------
   01001110     = 78
```

The product of two n-bit numbers needs up to 2n bits. 4 × 4 bits → 8 bits; 32 × 32 → 64 bits. That's why MCycle has two outputs: **Result2** = upper 32 bits ("high word"), **Result1** = lower 32 bits ("low word").

Names: the number being multiplied is the **multiplicand** (we call it M); the one we read bit by bit is the **multiplier** (Q). Each shifted copy we add is a **partial product**.

6.3 The shift-and-add multiplier (the template's design)

The template's MCycle does the binary long multiplication **one multiplier bit per clock cycle**:

- `shifted_op1` holds the multiplicand and **shifts left 1** each cycle (so it's always aligned with the current bit).
- `shifted_op2` holds the multiplier and **shifts right 1** each cycle (so the current bit is always bit 0).
- `temp_sum` accumulates: **if bit 0 of shifted_op2 is 1, add shifted_op1**.

Trace of 13 × 6 (registers 8 bits wide):

Cycle	op2[0]	Action	temp_sum after	op1 next	op2 next
1	0	—	00000000	00011010	00000011
2	1	add 00011010	00011010	00110100	00000001
3	1	add 00110100	01001110	01101000	00000000
4	0	—	01001110	…	…

Result 01001110 = **78** ✓. **4 bits → 4 cycles; 32 bits → 32 cycles** for unsigned numbers.

6.4 The problem with negative numbers

Shift-and-add assumes **every multiplier bit is worth a positive amount** (+1, +2, +4, …). In a signed (two's complement) multiplier, the **top bit is worth a negative amount** (−8 in 4 bits, -2^{31} in 32 bits). Shift-and-add has no way to subtract, so it gets negative multipliers wrong.

6.5 The template's workaround: sign-extend to double width

Because the hardware keeps only the low 2n bits (arithmetic "modulo 2^{2n}", Chapter 1.6), you can **sign-extend both operands to 2n bits** and multiply them as if they were unsigned. The extra (wrong) high parts fall off the end, and the low 2n bits are exactly right.

Example, −3 × 5 with 4-bit operands → extend to 8 bits: 11111101 × 00000101:

Cycle	op2[0]	Action	temp_sum after
1	1	add 11111101	11111101
2	0	—	11111101
3	1	add 11110100	11110001
4–8	0	— (5 more cycles of nothing)	11110001

Result 11110001 = **−15** ✓. But it took **8 cycles** for 4-bit numbers, i.e. **2n**. For 32-bit signed numbers: **64 iterations** (65 cycles counting the write-back cycle), with a **64-bit adder** and two 64-bit shift registers. Many of those cycles do nothing useful.

> **This is the weakness our Task 3 enhancement attacks.** Booth's algorithm handles signed numbers directly in **n** iterations.

Recap: binary multiplication = add shifted copies where the multiplier has 1s; product is 2n bits; the template does 1 bit per cycle; for signed numbers it sign-extends to 2n bits → 2n iterations (64 for 32-bit).

Exercises 6:

1. How many bits is the product of two 16-bit numbers? 2. Multiply 0101 × 0011 by shift-and-add. 3. Why can't plain shift-and-add handle a negative multiplier? 4. How many iterations does the template need for a 32-bit signed multiply? Unsigned?

Answers 6: 1. 32. 2. bit 0 = 1: add 0101; bit 1 = 1: add 01010 → 0101 + 1010 = 1111 = 15 ✓. 3. It treats the top bit as $+2^{n-1}$, but in a signed number it's -2^{n-1}, and it can only add. 4. 64 (signed), 32 (unsigned).

Chapter 7 — Booth's algorithm (radix-2): our Task 3 enhancement

This is the most important chapter. The demo quiz will mostly be about this.

7.1 The idea, in decimal

What's 999 × 7? You wouldn't add 7 nine hundred and ninety-nine times; you'd do **1000 × 7 − 7 = 6993**: one subtraction and one "addition" (of 1000 × 7).

7.2 The same idea in binary: runs of 1s

A **run** of consecutive 1s in binary is "the next power of two minus the lowest one":

```
0111        = 1000 - 0001           (7 = 8 - 1)
0011 1100   = 0100 0000 - 0000 0100 (60 = 64 - 4)
```

So multiplying M by a run of 1s doesn't need one addition per 1. It needs:

- **subtract M at the position where the run starts** (its lowest 1), and
- **add M at the position just after the run ends**.

Inside a run of 1s, or inside a run of 0s, **do nothing**.

7.3 How the hardware finds the start and end of runs

Read the multiplier from right (bit 0) to left. At each step look at **two adjacent bits**: the current bit **Q_0** and the bit just to its right, **Q_{-1}**. At the very start there is no bit to the right, so **Q_{-1} starts as 0**.

Q_0	Q_{-1}	What it means	Action on A
0	0	middle of a run of 0s	nothing
1	**0**	this bit is 1, the one before was 0: **a run of 1s starts**	**A = A − M**
0	**1**	this bit is 0, the one before was 1: **a run of 1s just ended**	**A = A + M**
1	1	middle of a run of 1s	nothing

Memory aid: "1-0 = start = subtract. 0-1 = end = add."

7.4 Why this works for negative numbers (no extension needed)

A negative number like 1110 (−2) is a run of 1s that **reaches the top bit and never ends**. Booth subtracts M where the run starts (bit 1, worth 2) and never adds anything back, because there's no "end". Result: −2M. Exactly right!

The general proof (be able to say the idea): Booth uses the digit $d_i = Q_{i-1} - Q_i$ at each bit i (that's the table: 01 → +1, 10 → −1, 00 and 11 → 0). Add up $d_i \times 2^i$ over all bits. Each bit Q_i appears twice: as $-Q_i \cdot 2^i$ in its own digit, and as $+Q_i \cdot 2^{i+1}$ in the next digit up. Together: $+Q_i \cdot 2^i$, its normal value. **Except the top bit**, which has no digit above it, so it keeps only $-Q_{n-1} \cdot 2^{n-1}$: **the negative weight of the sign bit in two's complement**. So the Booth digits add up to exactly the signed value of the multiplier.

Check with 0101 (5), reading pairs (Q_i, Q_{i-1}) from the right: (1,0) → −1 × 1; (0,1) → +1 × 2; (1,0) → −1 × 4; (0,1) → +1 × 8. Total: −1 + 2 − 4 + 8 = **5** ✓.

7.5 The hardware: four registers

Register	Width (32-bit design)	Starts as	Job
M	33 bits	the multiplicand, sign-extended by 1 bit	the value added or subtracted (never changes)
A	33 bits	0	the running total ("accumulator"); ends as the **upper half** of the product
Q	32 bits	the multiplier	holds the bits not yet read; ends as the **lower half** of the product
Q_{-1}	1 bit	0	the bit most recently pushed out of Q

7.6 One step (= one clock cycle)

1. **Look** at $\{Q_0, Q_{-1}\}$.
2. **Add or subtract:** A = A + M (01), A = A − M (10), or leave A (00, 11). One 33-bit adder does both (subtract = add the flipped M plus 1).
3. **Shift right:** treat $\{A, Q, Q_{-1}\}$ as one long 66-bit string and shift it right by 1 bit, **copying A's sign bit** into the new leftmost position (arithmetic shift). So:
   - the lowest bit of A moves into the top of Q,
   - the lowest bit of Q moves into Q_{-1},
   - the old Q_{-1} falls off.

Repeat **32 times** (one per multiplier bit). The 64-bit product is then **{A[31:0], Q}**: A's lower 32 bits followed by Q.

Why shift the result right instead of shifting M left? In long multiplication each new partial product is one place further left. Shifting everything already accumulated one place *right* has the same relative effect, and it means the adder only ever touches the top part (A): **33 bits instead of 64**. The finished low bits of the product drift down into Q, taking the place of multiplier bits that have already been used. No space is wasted.

7.7 Worked example 1: −3 × 5 in 4 bits (on slide 4 — learn it)

Setup (4-bit operands, so A and M are 5 bits):

- M = −3 → 4 bits 1101 → sign-extended to 5 bits: **11101**
- −M = +3 = **00011** (what we add when subtracting M)
- Q = 5 = **0101**, A = **00000**, Q_{-1} = **0**

Step	Q_0 Q_{-1}	Action	A after add/sub	{A, Q, Q_{-1}} before shift	after shift: A	Q	Q_{-1}
1	1 0	A − M = 0 + 3	00011	00011 0101 0	00001	1010	1
2	0 1	A + M = 1 + (−3)	11110	11110 1010 1	11111	0101	0
3	1 0	A − M = −1 + 3	00010	00010 0101 0	00001	0010	1
4	0 1	A + M = 1 + (−3)	11110	11110 0010 1	11111	0001	0

Product = {A[3:0], Q} = **1111 0001** = **−15** ✓ (−128 + 64 + 32 + 16 + 1 = −15).

How to do a shift by hand (step 1): write 00011 0101 0. Move everything one place right, putting a copy of the leftmost bit (0) on the left and dropping the last bit: **0**0001 1010 1 → A = 00001, Q = 1010, Q_{-1} = 1. In step 2 the leftmost bit is 1, so a 1 is copied in: 11110 1010 1 → **1**1111 0101 0.

Practise this example on paper until you can do it without looking.

7.8 Worked example 2: 6 × −2 (negative multiplier)

M = 6 = **00110**, −M = **11010** (−6). Q = −2 = **1110**.

Step	Q_0 Q_{-1}	Action	A after add/sub	after shift: A	Q	Q_{-1}
1	0 0	nothing	00000	00000	0111	0
2	1 0	A − M = −6	11010	11101	0011	1
3	1 1	nothing	11101	11110	1001	1
4	1 1	nothing	11110	11111	0100	1

Product 1111 0100 = **−12** ✓. Only **one** subtraction: the run of 1s in 1110 starts at bit 1 and never ends (Chapter 7.4).

7.9 Worked example 3: the hard case −8 × −1

M = −8 = **11000** (5 bits), so −M = **01000** = +8. **This only fits because A and M have the extra 5th bit.** Q = −1 = **1111**.

Step	Q_0 Q_{-1}	Action	A after add/sub	after shift: A	Q	Q_{-1}
1	1 0	A − M = +8	01000	00100	0111	1
2	1 1	nothing	00100	00010	0011	1
3	1 1	nothing	00010	00001	0001	1
4	1 1	nothing	00001	00000	1000	1

Product 0000 1000 = **+8** ✓. With only 4-bit A and M, −M = −(−8) would wrap round to −8 and the answer would come out as −8: **wrong sign**. In 32 bits this is $-2^{31} \times -1$, which is in our tests, our demo program and the visualisation.

7.10 The four "why" questions (you must be able to answer these)

1. **Why does Booth work for negative multipliers?** The Booth digits (previous bit − current bit) add up to the signed value: every bit gets its normal weight, except the top bit, which keeps only its negative weight -2^{n-1}, exactly as two's complement defines it (7.4).
2. **Why are A and M 33 bits?** So that −M fits when M is the most negative number -2^{31} ($+2^{31}$ doesn't exist in 32 bits), and so the running total in A never overflows before the shift halves it. The extra bit is dropped at the end.
3. **Why must the shift be arithmetic?** A holds a signed running total. Shifting right halves it, and halving a negative number must keep it negative, so the sign bit is copied in.
4. **Where does the product end up, and why?** In {A[31:0], Q}. Every shift moves one finished product bit from A into Q while one used multiplier bit leaves Q into Q_{-1}.

7.11 Cycle count

- Cycle 1: MCycle loads the registers **and** does step 1 (both in the same cycle — Chapter 10).
- Cycles 2–32: steps 2–32.

- Cycle 33: no step; **Busy** is now 0 and the CPU writes the answer into the destination register.

32 iterations, 33 cycles. The template needed 64 iterations, 65 cycles. **The slides count iterations (64 → 32); the board counts cycles (65 → 33). Both are right — always say which one you mean.**

7.12 Why not use Booth for unsigned multiplication?

Booth treats the top bit as negative. Unsigned `0xFFFFFFFF` (4,294,967,295) would be treated as −1. To use Booth you'd have to add a 0 bit on top (33 bits) and do 33 iterations — not faster than the existing 32. So `mulhu` (unsigned) still uses the template's shift-and-add.

7.13 The cost

- **+99 flip-flops** = the new registers: A (33) + M (33) + Q (32) + Q_{-1} (1).
- **+163 LUTs** (Vivado): the 33-bit add/subtract logic, the choice between add/subtract/nothing, and the muxes that load and shift A, Q, Q_{-1}.
- The template's 64-bit multiplier is **still there**, because unsigned multiply uses it. That's why the total area goes **up** (~+33% LUTs, +37% FFs).
- So it's a **speed-for-area trade-off**: about 2× faster signed multiplication for about a third more hardware. The manual explicitly allows trade-offs.

Recap: runs of 1s → subtract at the start, add after the end; look at {Q_0, Q_{-1}}: 10 subtract, 01 add, 00/11 nothing; then arithmetic shift {A,Q,Q_{-1}} right by 1; 32 iterations; product {A[31:0], Q}; A and M 33 bits for the most negative M; 65 → 33 cycles for ~+33% area.

Exercises 7:

1. {Q_0, Q_{-1}} = 01. What happens? 2. Trace 5 × −3 (M = 00101, −M = 11011, Q = 1101). 3. Trace −4 × −4 (M = 11100, −M = 00100, Q = 1100). 4. Why 33-bit A? 5. How many cycles for a 32-bit `mul`? 6. Where do the 99 extra flip-flops come from?

Answers 7:

1. A run of 1s just ended: A = A + M, then shift.
2. Steps: 10 → A − M: A = 11011 → shift → A 11101, Q 1110, Q_{-1} 1. 01 → A + M: 11101 + 00101 = 00010 → shift → 00001, 0111, 0. 10 → A − M: 00001 + 11011 = 11100 → shift → 11110, 0011, 1. 11 → nothing → shift → 11111, 0001, 1. Product 1111 0001 = **−15** ✓.
3. 00 → shift → 00000, 0110, 0. 00 → shift → 00000, 0011, 0. 10 → A − M = 00100 → shift → 00010, 0001, 1. 11 → shift → 00001, 0000, 1. Product 0001 0000 = **+16** ✓.
4. So −M fits for M = -2^{31}, and A doesn't overflow.
5. 33 (32 iterations + 1 write-back cycle).
6. A 33 + M 33 + Q 32 + Q_{-1} 1 = 99.

Chapter 8 — Radix-4 Booth, and how to go even faster

You implemented radix-2, but a TA may ask "how would you make it faster?" The standard answer is radix-4.

8.1 The idea: two bits per cycle

Radix-2 reads **1** new multiplier bit per cycle. Radix-4 reads **2** new bits per cycle, so it needs half as many steps: **16 iterations** (17 cycles) for 32 bits.

It looks at **3 bits** at a time — the 2 new bits plus the last bit of the previous pair: (Q_1, Q_0, Q_{-1}) — and turns them into one digit from **−2 to +2**. Then it shifts by **2**.

8.2 The table (and how to rebuild it without memorising)

Rule: the left bit counts −2, the middle and right bits count +1 each.

$Q_1\ Q_0\ Q_{-1}$	Digit	Add to A	$Q_1\ Q_0\ Q_{-1}$	Digit	Add to A
000	0	nothing	100	−2	−2M
001	+1	+M	101	−1	−M
010	+1	+M	110	−1	−M
011	+2	+2M	111	0	nothing

Example: 101 → −2 + 0 + 1 = −1.

8.3 Why every option is cheap

- **2M is M shifted left by 1** — just wiring, no adder.
- Negatives use the same "flip and add 1" trick (XOR with the negate signal, carry-in 1).
- So each step still uses **one adder**, preceded by a small mux choosing 0, M or 2M.

Why not plain base 4 with digits 0–3? 3M = 2M + M is *not* a shift — it needs a second adder. Booth avoids the digit 3 by writing "3" as "4 − 1" (−1 here, +1 carried into the next pair).

8.4 Radix-4 = two radix-2 steps merged

The radix-4 digit is exactly two radix-2 Booth digits combined: $d = (Q_{-1} - Q_0) + 2 \times (Q_0 - Q_1) = -2Q_1 + Q_0 + Q_{-1}$. So **one radix-4 step gives exactly the same A and Q as two radix-2 steps**, and it inherits the proof that Booth handles negative numbers.

8.5 What changes in the hardware

	Radix-2 (ours)	Radix-4
Bits examined per step	2	3
Choices for the adder	0, ±M	0, ±M, ±2M
A width (32-bit)	33	34 (2M needs one more bit)
Shift per step	1	2
After the shift, Q_{-1} =	the bit shifted out	the higher of the 2 bits shifted out
Iterations / cycles	32 / 33	16 / 17

8.6 Worked example: −3 × 6 in 4 bits, radix-4

M = −3 = **111101** (6 bits, because A and M are w+2 bits in radix-4). Q = 6 = **0110**. **Two** steps.

Step	$Q_1\ Q_0\ Q_{-1}$	Digit	A + d×M	A after shift by 2	Q	Q_{-1}
1	1 0 0	−2	0 − 2(−3) = 6	000001	1001	1
2	0 1 1	+2	1 + 2(−3) = −5	111110	1110	0

Product {A[3:0], Q} = 1110 1110 = **−18** ✓.

8.7 Going further: "how would you get to 9 cycles?"

9 cycles = 8 iterations + 1 write-back → **4 bits per cycle**. Options:

1. **Two radix-4 steps per cycle** ("dual radix-4"), or **four radix-2 steps per cycle** (wrap our step in a `for` loop that runs 4 times — I tested this: 9 cycles, all tests pass).
2. **Radix-16 Booth**: digits −8…+8, needing 3M, 5M, 7M (not shifts — need extra adders or precomputation). Rarely worth it.

The catch: doing several steps in one cycle puts several **adders in a row** (Chapter 3.6). The critical path gets longer, so the clock may have to slow down. **Total time = cycles × clock period**, so fewer cycles only helps if the period doesn't grow as much.

- **Radix-4** is the sweet spot: half the cycles, still one adder per step (2M is a shift), so the clock barely changes.
- **Chaining** steps only pays off if the clock has spare time. In **our** single-cycle CPU it does: the clock period is set by the slowest instruction (a memory load through the whole CPU), which is much longer than one adder. So MCycle could chain 2 adders without slowing the CPU at all.
- **Carry-save adders** add three numbers into two with no carry propagation (about one gate delay); with them, two radix-4 steps can be combined with only one real adder at the end. Taken all the way you get a **Wallace tree** — a fast single-cycle multiplier, which is what real CPUs and the FPGA's DSP blocks use.

One-sentence answer: "Radix-4 halves the cycles at almost no clock cost because 2M is just a shift; beyond that you chain adders per cycle, which only pays off when the clock has slack — and in our single-cycle CPU it does, because memory access sets the clock, not MCycle."

Exercises 8:

1. What digit is 110? 011? 100? 2. Why is 2M cheap? 3. Why is 3M expensive? 4. How many iterations does radix-4 need for 32 bits? 5. Recode Q = 0111 into radix-4 digits.

Answers 8: 1. −1, +2, −2. 2. It's M shifted left by 1 (wiring). 3. 3M = 2M + M needs an extra adder. 4. 16. 5. Group $(Q_1,Q_0,Q_{-1}) = (1,1,0) \rightarrow -1$; group $(Q_3,Q_2,Q_1) = (0,1,1) \rightarrow +2 \rightarrow -1 + 2\times4 = 7$ ✓.

Chapter 9 — Division (Task 1): restoring division

9.1 Long division in decimal

To divide 13 by 3 by hand: bring digits down one at a time, ask "how many times does 3 go into this?", write that digit of the answer, subtract, carry the remainder on.

9.2 In binary each answer digit is just "yes or no"

Each quotient digit is 0 or 1, so each step asks only: **"is what I have so far ≥ the divisor?"**

- **Yes** → quotient bit 1, subtract the divisor.
- **No** → quotient bit 0, leave it.

9.3 The registers

- **R** = the remainder so far (starts at 0).
- **Q** = the dividend, which gets replaced bit by bit by the quotient.
- **D** = the divisor.

In MCycle, R and Q sit side by side in one register, `temp_sum = {R, Q}`.

9.4 One step (= one clock cycle)

1. **Shift {R, Q} left by 1.** The top bit of Q (the next dividend bit) moves into the bottom of R; an empty slot appears at the bottom of Q.
2. **Try R − D.**
3. If R − D ≥ 0: keep it as the new R, and put **1** in the empty slot of Q.
4. If R − D < 0: **restore** — keep the old R — and put **0** in the slot.

After n steps: **Q = quotient, R = remainder**. It's called **restoring** division because a failed subtraction is undone (the old R is restored).

9.5 Worked example: 13 ÷ 3 in 4 bits

Start: R = 0000, Q = 1101 (13), D = 0011 (3).

Step	R after shift	R − 3	Decision	New R	New Q
1	1	−2	restore, q = 0	0001	1010
2	3	0	keep, q = 1	0000	0101
3	0	−3	restore, q = 0	0000	1010
4	1	−2	restore, q = 0	0001	0100

Quotient 0100 = 4, remainder 0001 = 1. Check: 4 × 3 + 1 = 13 ✓. Watch the quotient bits enter Q from the right: 0, 1, 0, 0.

9.6 Why R needs an extra bit while comparing

After the shift, R = 2 × (old R) + (new bit). Old R was less than D, so the new R can be up to 2D − 1, which can need one more bit than D. So the trial subtraction is done one bit wider. (After the decision, R is always less than D again, so it fits back in n bits.)

9.7 Signed division

Restoring division only works on non-negative numbers. For signed numbers:

1. Take the **magnitudes** (sizes): |dividend|, |divisor|.
2. Divide them as above.
3. **Quotient** is negative if the signs differ. **Remainder** takes the sign of the **dividend**.

This gives "rounding towards zero", which RISC-V requires, and always satisfies **dividend = quotient × divisor + remainder**:

a ÷ b	magnitudes	quotient	remainder	check
7 ÷ 3	2 r 1	2	1	2×3 + 1 = 7
−7 ÷ 3	2 r 1	**−2**	**−1**	−2×3 − 1 = −7
7 ÷ −3	2 r 1	**−2**	**1**	−2×−3 + 1 = 7
−7 ÷ −3	2 r 1	**2**	**−1**	2×−3 − 1 = −7

Worked: **−7 ÷ 2**: magnitudes 7 ÷ 2: steps give Q = 0011 (3), R = 0001 (1). Signs differ → quotient −3; dividend negative → remainder −1. Check: −3 × 2 − 1 = −7 ✓.

9.8 The two special cases

- **$-2^{31} \div -1$**: the true answer $+2^{31}$ doesn't fit in 32 bits. RISC-V defines the result as **-2^{31}, remainder 0**. Our hardware produces it naturally: $|-2^{31}| =$ `0x80000000`, which as an *unsigned* magnitude is 2^{31}; $\div 1 =$ `0x80000000`; the signs are equal so nothing is negated; read as signed that's -2^{31}. ✓
- **$x \div 0$**: the manual lets us assume this never happens, but we still match RISC-V: every trial subtraction $R - 0$ succeeds, so the quotient is **all ones (−1)** and the remainder is **x**. For a negative x, the quotient is deliberately *not* negated, so it stays −1 as RISC-V specifies.

9.9 Cycle count

One quotient bit per step → **32 iterations, 33 cycles**, for both `div` and `divu`.

Recap: shift {R,Q} left; try R − D; keep (q=1) or restore (q=0); n steps; signed = divide magnitudes, quotient negative if signs differ, remainder has dividend's sign; $-2^{31} \div -1 = -2^{31}$ r 0; $x \div 0 = -1$ r x.

Exercises 9:

1. Divide 6 ÷ 4 in 4 bits. 2. What is −7 ÷ 3 (quotient and remainder)? 3. Why "restoring"? 4. What is 5 ÷ 0 in RISC-V?

Answers 9: 1. Steps: restore, restore, restore, keep → Q = 0001 (1), R = 0010 (2) ✓. 2. −2 remainder −1. 3. A failed trial subtraction is undone. 4. Quotient −1 (all ones), remainder 5.

PART 3 — OUR DESIGN

Chapter 10 — The MCycle unit (`MCycle.v`)

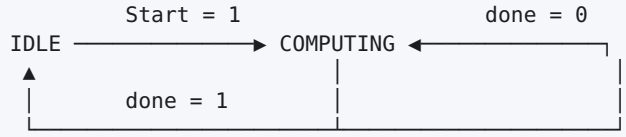
MCycle is the multi-cycle multiply/divide unit. It takes two 32-bit operands, works for 32 cycles, and produces two 32-bit results.

10.1 Its pins

Pin	Direction	Meaning
`CLK`, `RESET`	in	clock and reset (from the CPU)
`Start`	in	"please start an operation" (from the Decoder)
`MCycleOp` [1:0]	in	**00** signed multiply, **01** unsigned multiply, **10** signed divide, **11** unsigned divide
`Operand1`	in	multiplicand / dividend (rs1)
`Operand2`	in	multiplier / divisor (rs2)
`Result1`	out	low 32 bits of the product / the quotient
`Result2`	out	high 32 bits of the product / the remainder
`Busy`	out	1 while working — makes the CPU wait

Two parameters: `width` (4 for unit tests with small numbers, 32 in the CPU) and `BOOTH` (1 = our Booth signed multiply; 0 = the template's original, kept so the before/after can be compared).

10.2 The state machine: IDLE and COMPUTING



- **IDLE**: waiting. If Start becomes 1, go to COMPUTING and **raise Busy immediately**.
- **COMPUTING**: one iteration per cycle. When the last iteration has been done (`done` = 1), go back to IDLE and drop Busy.

Busy is a Mealy output (Chapter 3.7): in IDLE it becomes 1 *in the same instant* Start becomes 1, without waiting for a clock edge. This is essential: the CPU must freeze before the next clock edge (Chapter 11).

10.3 The "next value" coding style

Every register `x` has a partner variable `n_x`, meaning "what x will become at the next clock edge". The code works in two blocks:

1. **`always @(*)` (combinational):**
   - First, every `n_x = x` ("hold, unless told otherwise"). This default is also what prevents latches.
   - Then the logic modifies the `n_` values step by step, like a little program: load the operands, do one iteration, update the counter, set the results.

2. `always @(posedge CLK)` **(registers):** at the clock edge, every register copies its `n_` partner, all at once.

Why it's useful: several dependent steps can happen in one cycle. In the very first cycle, the code first **loads** the operands into the `n_` variables, and the very next lines **already perform iteration 1** on those freshly loaded values. In hardware that's just a chain of logic: load-mux → adder → shift → register input.

10.4 Starting an operation

```
if( RESET | (n_state == COMPUTING & state == IDLE) ) begin
    ... load everything ...
end
```

`n_state == COMPUTING & state == IDLE` means "we're in IDLE and about to start". **This is the only moment the operands are read.** MCycle copies them into its own registers (for Booth: A = 0, M = sign-extended Operand1, Q = Operand2, Q_{-1} = 0; for division: R = 0, Q = |dividend|, D = |divisor|, plus the two sign flags). After that it never looks at Operand1/Operand2 again — which matters when the destination register is also a source (Chapter 11.6).

10.5 The guard: no work in the last cycle

```
if( n_state == COMPUTING ) begin ... one iteration ... end
```

An iteration only runs if MCycle will still be computing after this edge. In the cycle in which Busy falls (state = COMPUTING, done = 1, so n_state = IDLE), **nothing changes** — the results are stable. That's the cycle the CPU uses to write the result into the register file.

10.6 Counting: `count` and `done`

`count` counts iterations 0, 1, 2, ... On the last one (count = 31 for 32-bit), the code sets `n_done = 1`. `done` is a register, so it becomes 1 at the next edge, and then Busy drops.

10.7 Timeline of one 32-bit Booth multiply

Cycle	State	Busy	What happens
1	IDLE (Start = 1)	1	operands loaded **and** iteration 1 done; stored at the edge
2	COMPUTING	1	iteration 2
…	COMPUTING	1	…
32	COMPUTING	1	iteration 32 (the last); done set
33	COMPUTING, done = 1	**0**	no iteration; results ready; **the CPU writes rd and moves on**
34	IDLE	depends on the next instruction	

Latency = iterations + 1 = 33 cycles (Booth multiply, unsigned multiply, division). The template's signed multiply: 64 + 1 = 65.

10.8 Code map (for "show me where…")

Lines	What
65–66	parameters `width`, `BOOTH`
87–88	last-iteration numbers: width−1 (Booth, unsigned mul, division), 2·width−1 (original signed mul)
130–151	`IDLE_PROCESS`: the state machine and Busy
154–176	start of `COMPUTING_PROCESS`: hold defaults for every `n_` variable (no latches), signs and magnitudes for division
182–199	starting an operation: load the template registers, the **Booth registers (187–190)**, the division registers and sign flags (193–198)
202	the guard (10.5)
203–225	**radix-2 Booth signed multiply** (10.9)
226–242	the template's shift-and-add: unsigned multiply (and signed when BOOTH = 0)
243–264	**restoring division** (10.10)
269–284	`STATE_UPDATE_PROCESS`: every register updated at the clock edge

10.9 The Booth code, line by line

```
if( (MCycleOp == 2'b00) & (BOOTH != 0) ) begin
```

Only signed multiply (op 00) uses Booth, and only when the `BOOTH` parameter is 1.

```
    case( {n_booth_Q[0], n_booth_Qminus1} )
        2'b01:   n_booth_A = n_booth_A + n_booth_M ;   // end of a run of 1s: add M
        2'b10:   n_booth_A = n_booth_A - n_booth_M ;   // start of a run of 1s: subtract M
        default: ;                                     // 00 / 11: nothing
    endcase
```

The decision table (Chapter 7.3). Synthesis builds A − M as A + (flipped M) + 1, so one adder does both.

```
    booth_shift = { n_booth_A, n_booth_Q, n_booth_Qminus1 } ;
    booth_shift = { booth_shift[2*width+1], booth_shift[2*width+1:1] } ;
```

Glue A, Q, Q_{-1} into one 66-bit string; shift right by 1, copying the top (sign) bit. Written out explicitly because Verilog's `>>>` only sign-extends if the value is declared `signed`.

```
    n_booth_A = booth_shift[2*width+1:width+1] ;
    n_booth_Q = booth_shift[width:1] ;
    n_booth_Qminus1 = booth_shift[0] ;
```

Cut the shifted string back into A (top 33 bits), Q (next 32), Q_{-1} (last bit).

```
    if( n_count == COUNT_LAST_W[7:0] ) n_done = 1'b1 ;   // 31: the last of 32 iterations
    n_count = n_count + 1 ;
    n_Result2 = n_booth_A[width-1:0] ;   // high word: A without its extra bit
    n_Result1 = n_booth_Q ;              // low word: Q
end
```

Count, flag the last iteration, and output {A[31:0], Q}.

10.10 The division code, line by line

```
div_shifted = { n_temp_sum, 1'b0 } ;
```

Shift {R, Q} left by 1 — into a vector one bit wider, so the bit leaving R isn't lost (R can reach 2D − 1, Chapter 9.6).

```
div_diff = { 1'b0, div_shifted[2*width : width] } - { 2'b00, n_shifted_op2[width-1:0] } ;
```

Trial subtraction R − D, done 2 bits wider than D so the top bit is a reliable sign (1 = negative = R < D).

```
if( ~div_diff[width+1] )   // R >= D
    n_temp_sum = { div_diff[width-1:0], div_shifted[width-1:1], 1'b1 } ;  // keep difference, q =
1
else                       // R < D
    n_temp_sum = { div_shifted[2*width-1:1], 1'b0 } ;                     // restore, q = 0
```

Keep or restore, and insert the quotient bit.

```
n_Result1 = n_neg_quotient  ? -quotient  : quotient ;
n_Result2 = n_neg_remainder ? -remainder : remainder ;
```

Apply the signs remembered at the start: `neg_quotient = (dividend negative XOR divisor negative) AND divisor ≠ 0`; `neg_remainder = dividend negative`.

Recap: two states; Busy rises instantly with Start; `n_` variables compute the next values, the clock edge stores them; operands are read only at the start; the last cycle does no work (write-back); 32 iterations + 1 = 33 cycles.

Chapter 11 — Connecting MCycle to the CPU (Task 2)

11.1 The M instructions

All use opcode **0110011** with **funct7 = 0000001**; funct3 picks the operation:

funct3	Instruction	Computes	MCycleOp	Result used
000	`mul`	low 32 bits of a × b	00	Result1
001	`mulh`	high 32 bits, signed × signed	00	Result2
010	`mulhsu`	high, signed × unsigned	(not supported → does nothing)	—
011	`mulhu`	high 32 bits, unsigned × unsigned	01	Result2
100	`div`	signed quotient	10	Result1
101	`divu`	unsigned quotient	11	Result1
110	`rem`	signed remainder	10	Result2
111	`remu`	unsigned remainder	11	Result2

The manual required `mul`, `div`, `divu`. `mulh`, `mulhu`, `rem`, `remu` were almost free (MCycle already computes both halves), so we added them.

Why `mul` doesn't care about signed vs unsigned: the low 32 bits of a product are the same either way (arithmetic modulo 2^{32}, Chapter 1.6). Only the high half differs — hence `mulh` vs `mulhu`.

11.2 Decoder changes (`Decoder.v`)

The old decoder looked only at funct7 bit 5 for R-type instructions, so `mul` would have been decoded as `add`. Now, if funct7 = 0000001, it sets four new control signals:

- **MStart** → MCycle's Start
- **MCycleOp** → which operation
- **MCycleSel** → "write MCycle's result instead of the ALU's"
- **MResultSel** → Result1 (low word/quotient) or Result2 (high word/remainder)

All default to 0, so no other instruction can start MCycle by accident.

11.3 The datapath (`RV.v`)

```
MCycle #(.width(32)) MCycle1( ..., .Operand1(RD1), .Operand2(RD2), ... ) ;   // operands = rs1,
rs2
assign MCycleResult = MResultSel ? MCycleResult2 : MCycleResult1 ;
assign ExecResult   = MCycleSel  ? MCycleResult  : ALUResult ;   // the new multiplexer
assign Result       = MemtoReg   ? ReadData      : ExecResult ;
```

11.4 The stall: `WE_PC = ~Busy`

A single-cycle CPU expects every instruction to finish in one cycle; `mul` takes 33. So the CPU **freezes**:

- While Busy = 1, the **PC doesn't change**, so instruction memory keeps outputting the **same instruction**.
- So the Decoder keeps producing the same control signals, and **Start stays 1**.
- Everything waits on the `mul` until Busy drops.

Because Busy is a Mealy output, it is already 1 during the very first cycle of the `mul`, before the clock edge — otherwise the PC would move on immediately.

11.5 Writing the result once: `WE = RegWrite & ~Busy`

During the stall the Decoder still says "write a register" (it's still a `mul`). Without the `& ~Busy`, partial results would be written every cycle. With it, the register is written **exactly once**, in the cycle Busy

falls — when the results are final — at the same clock edge where the PC moves to the next instruction. From the program's point of view, it looks like a normal instruction that just took longer.

11.6 Same register as source and destination

`div a6, a6, a1` reads a6 and writes a6. It works because MCycle copies its operands in the first cycle and never reads them again, and a6 isn't written until the end. The demo program tests this case.

11.7 Two M instructions in a row

Cycle	PC at	MCycle	Busy	Register write
1	`mul`	starting	1	—
2–32	`mul`	computing	1	—
33	`mul`	done	0	**t0 written**
34	`mulh`	starting again	1	—
…				
66	`mulh`	done	0	**t1 written**

In cycle 33 Start is still 1, but the state machine goes back to IDLE regardless; in cycle 34 the new instruction starts a fresh operation.

11.8 Memory size (`Wrapper.v`)

The demo program needed more space than the default 128 words, so instruction and data memories were enlarged to 256 words each (`IROM_DEPTH_BITS = DMEM_DEPTH_BITS = 10`).

Recap: M instructions = opcode 0110011 + funct7 0000001; the Decoder raises Start; `WE_PC = ~Busy` freezes the CPU; `WE = RegWrite & ~Busy` writes once at the end; a mux chooses ALU or MCycle.

Chapter 12 — The demo program (`test.asm`) and the board

12.1 What the program does

1. **Self-check.** For 16 test cases (a, b) it runs all seven M instructions and compares every result with a table of correct answers. Each LED is one instruction; any wrong result switches that LED off:
   - LED0 mul, LED1 mulh, LED2 mulhu, LED3 div, LED4 divu, LED5 rem, LED6 remu, **LED7 = all correct**.
   - **A correct CPU shows all 8 LEDs on (0xFF).**
2. **Cycle measurement.** It reads the hardware cycle counter, runs one `mul`, reads it again, and subtracts (also subtracting the cost of the reading itself). Result: **mul = 33, div = 33** cycles.
3. **Display loop** (forever): shows a value on the 7-segment display chosen by the switches.

12.2 The test cases (chosen to hit the tricky spots)

Case	a	b	Why
0–3	±7	±3	every sign combination
4	-2^{31}	−1	division overflow
5	-2^{31}	1	most negative number
6	$2^{31}-1$	$2^{31}-1$	biggest positive product
7	−1	−1	signed 1 vs unsigned huge
8 / 9	0 / 5	5 / 0	zero dividend / divide by zero
10, 11	±123456789	1000	ordinary numbers
12	0x12345678	0x9ABCDEF0	large mixed
13	−2	2	`divu` of a "negative"
14	1	0x80000000	divisor bigger than dividend
15	0xDEADBEEF	0x00010001	bit pattern

The table of correct answers was computed with the RISC-V rules and checked against RARS (the RISC-V simulator): RARS agrees with every entry.

12.3 Using the board

Control	Meaning
BTND	reset → reruns the self-check
SW15 = 0	browse mode: **SW3–0** = test case (0–F), **SW7–4** = what to show: 0 a, 1 b, 2 mul, 3 mulh, 4 mulhu, 5 div, 6 divu, 7 rem, 8 remu, **9 mul cycles**, **A div cycles**, B LED value
SW15 = 1	own numbers: **SW14–8** = a, **SW6–0** = b (7-bit signed, −64…63). Shows a × b; hold **BTNL** for a ÷ b, **BTNC** for a rem b, **BTNR** for divu
BTNU	pause the CPU clock
LED15–9 / LED8	the PC (too fast to read) / the CPU clock

12.4 Values to expect

Show	Switches	Display
−7 × 3	SW3–0 = 1, SW7–4 = 2	`FFFFFFEB` (−21)
$-2^{31} \div -1$	SW3–0 = 4, SW7–4 = 5	`80000000`
5 ÷ 0	SW3–0 = 9, SW7–4 = 5 / 7	`FFFFFFFF` / `00000005`
mul cycles	SW7–4 = 9	`00000021` (33)
−5 × 3 (own)	SW15 = 1, SW14–8 = 1111011, SW6–0 = 0000011	`FFFFFFF1` (−15); BTNL: `FFFFFFFF` (−1); BTNR: `55555553`

Chapter 13 — How we know it works (testing)

13.1 Self-checking testbenches

A testbench feeds inputs and checks outputs. **Self-checking** means it computes the right answer itself (using Verilog's `*`, `/` and `%` — allowed in testbenches because they're never turned into hardware) and prints PASS/FAIL. So thousands of cases can be checked without looking at waveforms.

13.2 `test_MCycle.v` (the MCycle unit alone): 76,083 checks

- **Exhaustive at 4 and 6 bits:** every possible pair of operands for all four operations. The algorithm is the same at every width (only the parameter changes), so proving it for *every* input at small widths is strong evidence for 32 bits.
- **32 bits:** corner values (0, ±1, ±2, most negative, most positive, bit patterns) against each other, plus 3,000 random pairs per operation.
- Also checks: the **exact cycle count**; Busy rising in the same cycle as Start; Start held high back-to-back; gaps between operations (results must hold); **operands changing while busy** (must be ignored); **reset in the middle**; and both BOOTH = 1 and 0.
- **Mutation testing:** we deliberately broke the design in many ways (wrong add/subtract pair, logical instead of arithmetic shift, M not sign-extended, wrong product bits, one iteration short, …). The testbench failed every time — proof it actually catches bugs.

13.3 `test_Wrapper.v` (the whole CPU + program): 229 checks

Runs the real program: checks the LEDs reach 0xFF, every display value for all 16 cases, the measured cycle counts, and own-number mode with each button. Plus `cpu_check.py`, the Assignment 2 test: 6,703 checks proving adding MCycle broke nothing.

13.4 Post-synthesis simulation

Runs the same MCycle tests on the **synthesised netlist** (the LUTs and flip-flops Vivado produced), proving the real circuit behaves like the Verilog. Both versions (BOOTH = 1 and 0) pass: signed multiply 33 and 65 cycles.

13.5 Hin Lok's testbench (the numbers on slide 4)

Slide 4's "514 / 467 checks" come from Hin Lok's own testbench. Our repository's `test_MCycle.v` is the larger one above. Make sure the testbench you submit is the one the slide quotes (or quote both).

Chapter 14 — Task 4: the visualisation

14.1 What was required

A **single HTML file**, made with AI help, that teaches our enhancement to someone who doesn't know it: enter numbers, step cycle by cycle, see the registers and datapath, the operation each step, the state changes and the cycle count. It must show **our** design. We must submit the **complete prompt history**. We must understand the algorithm (not the JavaScript).

14.2 `booth_visualisation.html` (Hin Lok's, enhanced)

Open it in a browser. It shows:

- inputs: width (4/8/32 bits), M and Q (decimal or hex like `0xFFFFFFFD`);
- **Next clock edge** = one iteration; Run; Reset; "Try minimum negative × −1"; export CSV;
- the counts: iterations done; template vs Booth iterations/cycles (e.g. 64/65 vs 32/33); State/Busy;
- registers M, A, Q, Q_{-1} bit by bit — **yellow** = changed this edge; **teal outline** = the two bits deciding the next step;
- the decision table with the next row highlighted;
- a **datapath diagram**: M → (+M / −M / 0) → adder ← A → arithmetic shift → A, Q, Q_{-1}, with the path used this cycle in teal and live values on the wires;
- the full trace table and the final product.

It was checked against the Verilog: A, Q and Q_{-1} after every step are identical to a simulation of MCycle.v.

In the demo: load −3 × 5 at 4 bits, press "Next clock edge" 4 times, narrating each step (Chapter 7.7). Then "Try minimum negative × −1" to show why the extra bit is needed.

14.3 The prompt history

`prompt_history.md`: Part 1 = Hin Lok's prompts (he must paste them in, with his GUIDE.md); Part 2 = the enhancement prompt. The submission needs the **complete** history.

Chapter 15 — Results, and our slides explained

15.1 The numbers

Metric (32-bit, Vivado, slide 5)	Baseline	Booth	Change
Signed multiply iterations	64	32	**−50%**
(cycles including write-back)	(65)	(33)	
Slice LUTs	487	650	+33.5%
Flip-flops	266	365	+37.2%
DSP blocks	0	0	— (no `*`)
Latches	0	0	—
Worst setup slack	+0.442 ns	+0.496 ns	both meet 100 MHz
Worst hold slack	−1.079 ns	−1.117 ns	normal after synthesis

15.2 Slide 2 — "Radix-2 Booth halves signed multiplication cycles"

The baseline sign-extends both operands to 64 bits and does 64 shift-and-add iterations; Booth handles signs directly in 32 iterations. **Say:** "Shift-and-add can only add, so the template handles negative numbers by extending to 64 bits — 64 iterations. Booth recodes the multiplier into −1/0/+1 digits that already include the sign bit's negative weight, so it needs only the 32 real bits. Unsigned multiply stays at 32 iterations, and there's no `*` anywhere."

15.3 Slide 3 — "How radix-2 Booth updates the hardware"

The four registers, the loop (inspect → add/sub/hold → arithmetic shift → repeat 32×), the table, the product {A[31:0], Q}, why 33 bits. **Say:** "10 is the start of a run of 1s, so subtract M; 01 is the end, so add M; 00 and 11 are inside a run, so nothing. Then shift A, Q and Q_{-1} right together, arithmetically, because A is signed. The extra bit lets −M exist when M is -2^{31}."

15.4 Slide 4 — "Verified against expected products and the baseline"

The −3 × 5 trace (Chapter 7.7) and the verification counts. **Say:** "This is the exact register trace — the visualisation shows it step by step." Be ready to do it on paper.

15.5 Slide 5 — "Fewer cycles at a higher hardware cost"

Say: "Half the signed-multiply cycles for about a third more LUTs and flip-flops. The 99 extra flip-flops are exactly the Booth registers: 33 + 33 + 32 + 1. The extra LUTs are the 33-bit adder/subtractor and the shift and load muxes; the template's 64-bit multiplier is still there because unsigned multiply uses it. No DSPs because no `*`; no latches because every signal has a default."

If asked about slack: "Setup slack is positive, so both meet a 100 MHz clock. Negative hold slack after synthesis is normal — there's no real wiring yet; place-and-route fixes hold. It's the same in both designs, so not caused by Booth."

15.6 Two things to fix or footnote before the demo

1. **The slide 5 numbers were measured on Hin Lok's MCycle, in which division was still the template's placeholder.** The final MCycle adds the same division hardware to both, so the *differences* (+163 LUTs, +99 FFs) still hold but the *percentages* will be smaller. Re-run Vivado on the final file with BOOTH = 1 and 0, or add a footnote: "measured with the division placeholder, to isolate the multiplier".
2. **Slide 4's check counts** come from Hin Lok's testbench — submit it, or quote ours.

PART 4 — THE DEMO

Chapter 16 — The demo: what to expect and what to do

16.1 What it's like

From a group that already demoed (second-hand): **no debugging** (you show a working design, you don't fix things live), **the same format as earlier demos**, and **you'll be quizzed on Booth**. Everyone is scored individually, so **every member must be able to explain everything**, including parts they didn't write. The manual allows **one program and one bitstream**, and the files must be **exactly** the submitted ones.

16.2 Before you go

1. **Bitstream** rebuilt with the final MCycle.v (Booth on), `TOP` as top, constraints file enabled. Program the board once: 8 LEDs on, view 9 shows `00000021`.
2. **On the laptop, ready to show:**
   - Vivado with `test_MCycle` and `test_Wrapper` already run (`run all`): **PASSED – all 76083 checks** and **229 passed, 0 failed**.
   - The post-synthesis simulation and utilisation for both versions (for the slide numbers).
   - The slides.
   - `booth_visualisation.html` in a browser.
   - `MCycle.v` open at the Booth block (lines 203–225) and the division block (243–264).
3. **Submission uploaded** (Chapter 16.5).

16.3 A demo script (5–8 minutes)

#	Show	Say
1	Board after programming: **all 8 LEDs on**	"On reset the program runs mul, mulh, mulhu, div, divu, rem and remu on 16 tricky test cases and checks every answer. Each LED is one instruction; all on means all 112 results are correct."
2	SW3–0 = 1, SW7–4 = 2 → `FFFFFFEB`	"Case 1 is −7 × 3 = −21, computed live by the `mul` instruction."
3	SW3–0 = 4, SW7–4 = 5 → `80000000`	"$-2^{31} \div -1$ — the overflow case. RISC-V says -2^{31} remainder 0, and that's what we get."
4	SW3–0 = 9, SW7–4 = 5 → `FFFFFFFF`, 7 → `00000005`	"5 ÷ 0: we may assume it never happens, but we still give RISC-V's answer: −1 remainder 5."
5	SW7–4 = 9 → `00000021`	"The hardware cycle counter measured one `mul`: 33 cycles. With the original multiplier it was 65."
6	SW15 up, TA's numbers	"Any 7-bit numbers: × by default; hold BTNL ÷, BTNC remainder, BTNR unsigned divide."
7	Slides 2–5	Chapter 15.
8	Visualisation, −3 × 5 at 4 bits	Step 4 times, narrating (Chapter 7.7).
9	Simulations	Both testbenches pass; post-synthesis too.

16.4 If a question stumps you

Go back to basics out loud: "Booth looks at the current bit and the previous one…" and work an example on paper. Showing you can reason is worth more than a memorised answer.

16.5 Submission checklist (from the manual)

Zip named `GroupXX_Monday/Friday_Asst3.zip` (your Assignment 2 folder says Friday group 39 → probably `Group39_Friday_Asst3.zip`). One per group. Only these files — not the whole project:

Required	Our files
.v files created/modified + testbenches	`MCycle.v`, `RV.v`, `Decoder.v`, `Wrapper.v`, `test_MCycle.v`, `test_Wrapper.v` (+ Hin Lok's testbench if the slides quote it)
.bit file	`TOP.bit` from the final build
.asm	`test.asm`
1–4 slides	`Task3_Booth_Slides.pdf`
visualisation	`booth_visualisation.html`
prompt history + Markdown files used	`prompt_history.md` (with Hin Lok's part filled in), his `GUIDE.md`

Chapter 17 — Question bank (with answers)

Cover the answer, say yours out loud, then check. The Booth radix-2 questions are the most likely.

Booth radix-2 (expect most questions here)

What is the problem Booth solves? The template's shift-and-add can only add, so for signed numbers it sign-extends to 64 bits and does 64 iterations. Booth handles the sign directly: 32 iterations.

Explain Booth's algorithm in one sentence. Scan the multiplier from the LSB, looking at each bit and the one to its right: at the start of a run of 1s subtract M, at the end of a run add M, otherwise do nothing; shift {A, Q, Q_{-1}} right arithmetically after each step; after w steps {A, Q} is the product.

What is Q_{-1} for? It remembers the bit that was just shifted out of Q, so that each step can see two adjacent bits and detect where a run of 1s starts or ends. It starts at 0 (an imaginary bit to the right of the LSB).

Why does 10 mean subtract and 01 mean add? A run of 1s from bit i to bit j equals $2^{j+1} - 2^{i}$. The 10 pair is at bit i (the run starts: $-M \cdot 2^{i}$), and the 01 pair is at bit j+1 (the run has ended: $+M \cdot 2^{j+1}$).

Why does Booth work for negative multipliers? The digit is $d_i = Q_{i-1} - Q_i$. Summing $d_i \cdot 2^i$, every bit gets its normal weight except the top bit, which keeps only -2^{w-1}: exactly the two's complement sign weight.

Why does the baseline take 64 iterations for signed but Booth 32? The baseline can only add positive multiples, so it sign-extends to 64 bits and processes all 64 bits. Booth handles the sign bit's negative weight in the recoding, so it only processes the 32 real bits.

Why is A 33 bits? So that $-M$ fits when $M = -2^{31}$, and so the running sum doesn't overflow before the shift. Bit 32 is discarded at the end.

Why must the shift be arithmetic? A holds a signed partial product. Shifting right halves it, and halving a negative number must keep it negative, so the sign bit is copied in.

Where does the product end up? {A[31:0], Q}. Q starts as the multiplier; each shift moves one product bit from A into Q and one used multiplier bit out into Q_{-1}.

How is A − M implemented? A + ~M + 1 (invert and add 1). Synthesis does this from `n_booth_A - n_booth_M`, using the adder's carry-in, so add and subtract share one adder.

What happens for 00 and 11? Nothing is added; A only shifts. 00 is inside a run of 0s, 11 inside a run of 1s.

Walk me through −3 × 5 / show me edge 2. Chapter 7.7. Edge 2: pair 01 → A = 1 + (−3) = −2 = 11110 → shift {11110, 1010, 1} → {11111, 0101, 0}.

What's the worst case for Booth? Is it ever slower? In our hardware it always takes exactly 32 iterations, whatever the operands, because every iteration does at most one add/subtract and one shift in one cycle. (In software or in variable-latency designs, alternating bits like 0101… are the worst case, because they need an add or subtract at every step. Here it doesn't matter.)

Why didn't you use Booth for unsigned multiplication? Booth reads the top bit as negative. For unsigned you'd extend to 33 bits and need 33 iterations: no faster than the existing 32.

How many extra flip-flops, and why? 99 = A 33 + M 33 + Q 32 + Q_{-1} 1.

How could you make it faster still? Radix-4 Booth: 2 bits per cycle with digits {−2, −1, 0, +1, +2}, so 16 iterations.

How could you make it smaller? Share Booth's 33-bit adder with unsigned multiply (extend unsigned operands by a 0 bit) and remove the 64-bit shift-and-add, or share one adder between multiply and divide.

Booth radix-4

What are the radix-4 digits and why? {−2, −1, 0, +1, +2}, from overlapping 3-bit groups: d = $-2 \cdot Q_{2i+1} + Q_{2i} + Q_{2i-1}$. All are cheap: 2M is a shift, negatives are invert + 1. Digit 3 is avoided because 3M needs an extra adder.

Why overlapping groups of 3? Each digit needs the previous pair's top bit (Q_{2i-1}) to know whether a run of 1s continues across the pair boundary, just as radix-2 needs Q_{-1}.

How many iterations, and what shift? w/2 = 16 for 32-bit, shifting right by 2 each time.

Why is A w+2 bits in radix-4? ±2M needs one more bit than M, and the sum needs headroom for its sign.

What's 100 / 011 / 111 / 101? 100 → −2M; 011 → +2M; 111 → 0; 101 → −M. (Left bit −2, middle and right +1 each.)

How is radix-4 related to radix-2? Each radix-4 digit is two radix-2 digits merged: d = $e_{2i} + 2e_{2i+1} = -2Q_{2i+1} + Q_{2i} + Q_{2i-1}$. One radix-4 step gives exactly the same A and Q as two radix-2 steps, so it inherits the signed-number proof.

Why can't radix-4 use digit 3? 3M = 2M + M isn't a shift; it would need a second adder or a precomputed 3M. Booth rewrites 3 as 4 − 1 (−1 here, +1 into the next group), so only 0, ±1, ±2 are needed.

After a radix-4 step, what goes into Q_{-1}? The higher of the two bits shifted out (old Q_1), because the next group overlaps this one in that bit.

How do you negate in radix-4? Pick |d|·M (M or M shifted left 1), then XOR every bit with the negate signal and add it as the carry-in: ~pp + 1.

Recode 0111 (radix-4). (1,1,0) → −1; (0,1,1) → +2: −1 + 2·4 = 7.

How many iterations for 32-bit unsigned in radix-4? Unsigned needs a 0 on top (33 bits), padded to an even 34 bits: 17 iterations (18 cycles). That's what our first version did.

Would radix-4 cost more hardware than your radix-2? Slightly more datapath: a 5-way choice (0, ±M, ±2M) instead of 3, a 3-bit decode, and A one bit wider. The registers are about the same. Our earlier radix-4 version was actually smaller than the baseline, because it replaced the 64-bit datapath for both signed and unsigned instead of adding beside it.

What about radix-8? Digits up to ±4 include ±3M, which needs an extra adder or a precomputed 3M. That's why radix-4 is the usual choice.

Division (Task 1)

Explain restoring division. {R, Q} = {0, dividend}. Each cycle: shift {R, Q} left by 1, compute R − D; if ≥ 0 keep it and set the new quotient bit to 1, else keep the old R ("restore") and set 0. After 32 cycles Q is the quotient and R the remainder.

Why "restoring"? When R − D is negative, the subtraction is thrown away and the original R is kept.

Why does the comparison need an extra bit? After the shift, R can be up to 2D − 1, which can need one more bit than D.

How do you handle signed division? Divide the magnitudes; negate the quotient if the signs differ; give the remainder the dividend's sign. That rounds the quotient towards zero, as RISC-V requires.

What is $-2^{31} \div -1$? -2^{31}, remainder 0 (RISC-V's defined overflow result). It falls out naturally: $|-2^{31}|$ = 0x80000000 read as unsigned is 2^{31}, ÷ 1 = 0x80000000, signs equal so no negation.

What about ÷ 0? Every trial subtraction succeeds, so the quotient is all ones and the remainder is the dividend: the RISC-V-defined result. The `& (|Operand2)` term stops the quotient being negated for a negative dividend.

How could division be faster? Non-restoring division (no restore mux), or radix-4 / SRT division (2 quotient bits per cycle).

MCycle and the CPU (Task 2)

How does the CPU wait for MCycle? `WE_PC = ~Busy`: while Busy the PC holds, so the same instruction stays in the decoder and Start stays high. Busy is combinational, so it rises in the same cycle as Start, before the PC can move.

When is the result written to the register file? In the cycle in which Busy falls (`WE = RegWrite & ~Busy`): the results are final, and at that clock edge rd is written and the PC advances, like a normal single-cycle instruction.

Why gate WE with ~Busy? So partial results are never written to rd, and the write happens exactly once.

What if rd = rs1 (e.g. `div a6, a6, a1`)? MCycle copies its operands into its own registers in the first cycle and never reads them again, and rd isn't written until the end, so it works. The demo program tests this.

What if two M instructions are back to back? In the cycle Busy falls, the FSM returns to IDLE regardless of Start; the next instruction then starts a new operation in the following cycle.

How are the M instructions decoded? Opcode 0110011 (R-type) with funct7 = 0000001; funct3 picks the operation. The Decoder sets MStart, MCycleOp, MCycleSel (write MCycle's result instead of the ALU's) and MResultSel (Result1 = low word / quotient, Result2 = high word / remainder).

Why does `mul` not care about signed vs unsigned? The low 32 bits of a product are the same either way (arithmetic modulo 2^{32}). Only the high word differs: hence `mulh` vs `mulhu`.

Which instructions did you implement? mul, div, divu (required) plus mulh, mulhu, rem, remu (free, because MCycle produces both words). mulhsu is not supported (decodes as a NOP).

Verilog, synthesis and testing

What is a latch and why is it bad? Memory created by accident when a signal in an `always @(*)` block isn't assigned on every path. It makes timing unpredictable. MCycle gives every combinational signal a default at the top of the block, so there are none (slide 5: 0 latches).

Why the `n_` variables? The combinational block computes what each register will be next, step by step; the clocked block copies them all in at the edge, in one place. It lets several dependent steps (load, then the first iteration) happen in one cycle.

What is a Mealy machine? One whose outputs depend on the inputs as well as the state. Busy depends on Start in the IDLE state, so it rises without waiting for a clock edge.

Behavioural vs post-synthesis simulation? Behavioural simulates the Verilog as written; post-synthesis simulates the netlist of LUTs and flip-flops synthesis produced. Running the same tests on both proves the synthesised circuit behaves like the code.

How do you know the testbench is good? It's self-checking against an independent reference (Verilog's own `*`, `/`, `%`, legal in a testbench), exhaustive at 4 and 6 bits, and deliberately broken designs (mutations) all make it fail.

What do setup and hold slack mean? Setup slack: how early the slowest signal arrives before the next clock edge (must be ≥ 0). Hold slack: how long a signal stays stable after the edge (must be ≥ 0). Post-synthesis hold values are estimates without routing and are fixed in implementation.

Two's complement quickies

What is `FFFFFFEE`? −18 (invert → `00000011`, +1 → `00000012` = 18). **What is `FFFFFFF1`?** −15. **Why does divu of −6 by 3 give `55555553`?** divu reads `FFFFFFFA` as 4,294,967,290; ÷ 3 = 1,431,655,763 = 0x55555553, remainder 1.

Other ways we could have improved MCycle (for "why Booth?")

Option	What it does	Effect
Radix-2 Booth (ours)	Signed multiplier recoded to {−1, 0, +1}	Signed mul 64 → 32 iterations; more area
Radix-4 Booth	2 bits per cycle, digits {−2…+2}	16 iterations
Sharing one adder between mul and div	One adder/subtractor serves both units	Less area, same speed
Non-restoring division	Never "restores": if R went negative, add D in the next step instead of subtracting	Removes the restore mux; same 32 cycles
SRT / radix-4 division	2 quotient bits per cycle	~16 cycles, much more complex
Early termination	Stop when the remaining multiplier bits are all 0s (or all 1s)	Fewer cycles for small multipliers; variable latency
`*` operator / DSP	Single-cycle multiplier in DSP blocks	**Forbidden** by the manual

Why radix-2 Booth: it directly attacks the baseline's biggest waste (64 iterations for signed numbers, because of the 64-bit sign extension), it's simple enough to prove and verify exhaustively, and it gives a clean, measurable 2× speed-up.

Chapter 18 — Practice problems (with answers)

Numbers

- **Convert:** 1001 0110 to decimal [150]; −6 to 4-bit two's complement [1010]; `0xFFFFFFFE` signed [−2]; `0x80000000` signed [−2,147,483,648] and unsigned [2,147,483,648].
- **Arithmetic shift:** 1000 `>>>` 1 [1100 = −4]; 0110 `>>>` 1 [0011].

Division (4-bit)

- **−7 ÷ 2:** magnitudes 7 ÷ 2: restore, restore, keep, keep → Q = 0011, R = 0001 → quotient **−3**, remainder **−1** (−3 × 2 − 1 = −7 ✓).
- **7 ÷ −2:** same magnitudes → quotient **−3**, remainder **+1** (dividend positive).
- **6 ÷ 4:** restore ×3, keep → quotient 1, remainder 2.

Booth

Do these on paper, then check them in `booth_visualisation.html` (radix-2) or against the answers.

Radix-2, 4-bit:

Problem 1 — 5 × −3. M = 00101, −M = 11011, Q = 1101.

Edge	pair	op	A before shift	A after	Q after	Q_{-1}
1	10	A − M	11011	11101	1110	1
2	01	A + M	00010	00001	0111	0
3	10	A − M	11100	11110	0011	1
4	11	hold	11110	11111	0001	1

Product 1111 0001 = **−15**.

Problem 2 — −4 × −4. M = 11100, −M = 00100, Q = 1100.

Edge	pair	op	A before shift	A after	Q after	Q_{-1}
1	00	hold	00000	00000	0110	0
2	00	hold	00000	00000	0011	0
3	10	A − M	00100	00010	0001	1
4	11	hold	00010	00001	0000	1

Product 0001 0000 = **+16**.

Problem 3 — 7 × −3. Edges: 10 A − M → 11100 / 1110 / 1; 01 A + M → 00001 / 1111 / 0; 10 A − M → 11101 / 0111 / 1; 11 hold → 11110 / 1011 / 1. Product 1110 1011 = **−21**.

Radix-4, 4-bit (A 6 bits):

Problem 4 — −5 × 7. M = 111011, Q = 0111.

Edge	bits	digit	A + d·M	A after	Q after	Q_{-1}
1	110	−1	0 + 5 = 5	000001	0101	1
2	011	+2	1 − 10 = −9	111101	1101	0

Product 1101 1101 = **−35**.

Problem 5 — 6 × 6. M = 000110, Q = 0110.

Edge	bits	digit	A + d·M	A after	Q after	Q_{-1}
1	100	−2	−12	111101	0001	1
2	011	+2	−3 + 12 = 9	000010	0100	0

Product 0010 0100 = **+36**.

Problem 6 — 7 × −3. Edges: 010 → +1: A = 7 → 000001 / 1111 / 0; 110 → −1: A = 1 − 7 = −6 → 111110 / 1011 / 1. Product 1110 1011 = **−21** (2 edges, against 4 in radix-2 for the same problem).

Quick recoding drills (answers in brackets):

- Radix-2 digits of 0111 (from the right): [−1, 0, 0, +1 → −1 + 8 = 7]
- Radix-2 digits of 1000: [0, 0, 0, −1 → −8]
- Radix-4 digits of 0111: [(1,1,0) → −1; (0,1,1) → +2 → −1 + 8 = 7]
- Radix-4 digits of 1000: [(0,0,0) → 0; (1,0,0) → −2 → −8]
- Radix-4 digits of 0101: [(0,1,0) → +1; (0,1,0) → +1 → 1 + 4 = 5]

Chapter 19 — Glossary

Term	Meaning
LUT (look-up table)	The FPGA's basic logic element: a small memory that can implement any function of up to 6 inputs
Flip-flop (FF)	A 1-bit register, updated at the clock edge
Slice	A group of LUTs and flip-flops in the FPGA
DSP block	A dedicated hardware multiplier/accumulator in the FPGA; `*` would use these
CARRY4	The FPGA's fast carry chain, used to build adders
Latch	Level-sensitive memory, usually created by mistake in combinational code
Synthesis	Turning Verilog into a netlist of LUTs and flip-flops
Implementation	Placing and routing that netlist onto the chip
Bitstream	The file that configures the FPGA with the implemented design
Behavioural simulation	Simulating the Verilog source
Post-synthesis simulation	Simulating the synthesised netlist
Setup / hold slack	Timing margins before / after the clock edge (positive = OK)
Mealy / Moore machine	FSM whose outputs depend on state and inputs / on state only
Blocking (`=`) / non-blocking (`<=`)	`=` updates immediately (used in combinational blocks), `<=` at the end of the time step (used for registers)
Two's complement	Signed number format where the top bit has negative weight
Sign extension	Widening a signed number by copying its top bit
Arithmetic shift right	Shift right copying the sign bit (signed ÷ 2, rounding down)
Latency	Number of cycles an operation takes
Radix	The number base: radix-2 processes 1 bit per step, radix-4 two bits
MCycleOp	00 signed mul, 01 unsigned mul, 10 signed div, 11 unsigned div
Busy	MCycle output: 1 while computing; stalls the CPU
RV32M	The RISC-V multiply/divide extension

Chapter 20 — One-page cheat sheet (read this in the morning)

Radix-2 Booth (signed mul, ours): A = 0 (33b), M = sign-ext multiplicand (33b), Q = multiplier (32b), $Q_{-1} = 0$. Each cycle: $\{Q_0,Q_{-1}\}$: 00/11 hold, 01 A += M, 10 A −= M; then $\{A,Q,Q_{-1}\}$ >>> 1 (copy sign). 32 iterations → product {A[31:0], Q}. 33 cycles with write-back (baseline 64 / 65). +99 FFs (33+33+32+1).

Why it works: $d_i = Q_{i-1} - Q_i$; $\Sigma\, d_i 2^i$ = signed value (top bit keeps -2^{w-1}). Runs of 1s: $2^{j+1} - 2^i$.

Radix-4: groups $(Q_{2i+1}, Q_{2i}, Q_{2i-1})$, left bit −2, others +1: 000/111 0, 001/010 +M, 011 +2M, 100 −2M, 101/110 −M. Choose 0/M/2M, negate by ~pp + 1; $\{A,Q,Q_{-1}\}$ >>> 2; new Q_{-1} = old Q_1; 16 iterations (17 cycles); A w+2 bits. One radix-4 step = two radix-2 steps. −3 × 6: 100 → A = +6 → 000001|1001|1; 011 → A = −5 → 111110|1110|0 → 1110 1110 = −18.

−3 × 5 (4-bit): M 11101, −M 00011, Q 0101. Edges: 10 A−M → 00001|1010|1; 01 A+M → 11111|0101|0; 10 → 00001|0010|1; 01 → 11111|0001|0. Product 1111 0001 = −15.

Restoring division: {R,Q} = {0, |dividend|}; shift left; R − D ≥ 0 ? (R = R − D, q = 1) : (restore, q = 0); 32 iterations; quotient negated if signs differ; remainder has the dividend's sign. $-2^{31} \div -1 = -2^{31}$ r 0; x ÷ 0 = −1 r x.

CPU: M instructions = opcode 0110011, funct7 0000001. WE_PC = ~Busy (stall); WE = RegWrite & ~Busy (write once, when Busy falls); result mux ALU / MCycle.

Board: LEDs 0xFF = self-check passed. SW15 = 0: SW3–0 case, SW7–4 view (2 mul … 9 mul cycles = `21`, A div cycles = `21`). SW15 = 1: SW14–8 a, SW6–0 b; × / BTNL ÷ / BTNC rem / BTNR divu. BTND reset.

Numbers: signed mul 64 → 32 iterations (65 → 33 cycles); Vivado (slide 5) 487 → 650 LUTs (+33.5%), 266 → 365 FFs (+37.2%), 0 DSP, 0 latches. Tests: 76,083 MCycle checks, 229 CPU checks, 6,703 RV32I checks.

Practice: Chapter 18. Check radix-2 traces in the visualisation and on the board.